

PART VI — THE SELF-LEARNING LAYER

Chapter 31

Learning From Failure, Part 2: User Thumbdowns

“Silence tells a system a query failed. A user's own words tell it why.”

Chapter 31 — Learning From Failure, Part 2: User Thumbdowns

Chapter 30 built a blocklist from the cheapest possible failure signal: silence, a retrieval that returned nothing. This chapter builds the richer, rarer signal — a user who noticed a wrong answer, cared enough to say so, and explained in their own words what was wrong. That explanation is qualitatively different from a zero-chunk retrieval, and it demands a correspondingly richer record and a more careful injection strategy than Chapter 30's simple blocklist.

This chapter walks through `'user_thumbdowns.json'` end to end: what a thumbdown actually captures, how the `'bad'` command gates that capture behind a minimum feedback length, how a prior thumbdown gets looked up and injected as a `'USER-FLAGGED PRIOR FAILURE'` block the next time a related query arrives, and a real worked example — this project's own ASD-disambiguation dry run — showing the mechanism catch and correct a genuinely wrong answer on the very next attempt.

By the end of this chapter you will be able to design a thumbdown record that captures not just that an answer was wrong but the full trajectory that produced it, explain why exact-match normalized-query lookup is a deliberate trade-off rather than an oversight, and recognize the difference between feedback a retrieval system can act on and feedback it structurally cannot.

31.1 The richer signal

A zero-chunk retrieval (Chapter 30) is an unambiguous failure — nothing came back, so nothing could have been used to answer the question. A thumbdown is a different, harder case: the answer was grounded in real, relevant-looking chunks, passed the JUDGE phase's automatic quality check, and was still wrong in a way only the user could recognize. `'check_answer_quality()'` verifies that an answer is **supported** by its retrieved evidence; it has no way to verify that the evidence itself was the **right** evidence for what the user actually meant.

This is precisely the gap `'run_batch.py'`'s Batch 1 is designed to exercise: six consecutive questions about "ASD" answered correctly and confidently as Autism Spectrum Disorder — grounded, well-cited, judge-approved — until the seventh question reveals the user meant Adjustable Speed Drives the entire time. Every one of those six answers was internally consistent and evidence-backed. All six were still wrong, and no automatic quality gate built from the retrieved evidence alone could have caught it.

31.2 The bad command and MIN_FEEDBACK_LEN

`'cmd_bad()'` is the entry point: it reads `'user_feedback'` from state, forwards it to `'feedback_store.mark_last_bad(feedback=feedback, variants=variants)'`, and reports back to the user based on what actually got persisted. The `'MIN_FEEDBACK_LEN = 10'` constant is the gate — feedback shorter than ten characters flips the last interaction's quality to `'USER_THUMBSDOWN'` but does **not** create a structured thumbdown record, because ten characters is not enough text to extract any usable signal from.

```
def cmd_bad(state: GraphState) -> dict:
    feedback = (state.get("user_feedback") or "").strip()
    variants = list(services.last_variants_with_chunks)
    ok = feedback_store.mark_last_bad(feedback=feedback, variants=variants)
    if not ok:
        print("Nothing to flag yet.\n")
        return {}
    if feedback and len(feedback) >= MIN_FEEDBACK_LEN:
        print("Last answer flagged as bad and persisted to
user_thumbsdowns.json.\n")
    elif feedback:
        print(f"Last answer flagged as bad. Feedback too short
(<{MIN_FEEDBACK_LEN} chars) — not persisted.\n")
    else:
        print("Last answer flagged as bad. No feedback provided.\n")
    return {}
```

This three-way branch is worth reading closely: a bare thumbsdown with no feedback text still matters (it flips the interaction's quality, keeping it out of Chapter 32's distillation pool), but only feedback meeting the length floor earns the richer treatment this chapter is actually about. Ten characters is deliberately low — "wrong topic" is nine characters and would just miss it, while "wrong answer" clears it — the floor exists to reject accidental or reflexive thumbs-down, not to demand a full paragraph.

31.3 What to capture per thumbsdown

A thumbsdown record captures substantially more than the feedback text alone: the original query, the answer judged bad, the user's feedback verbatim, and — critically — every query variant attempted during that run, each with the chunks it retrieved from both tracks. `\_append\_thumbsdown()` builds exactly this shape, truncating chunk content to 1,000 characters per chunk to keep the record bounded (Chapter 29.1B's sizing discipline applied here to a richer record).

Anatomy of one user_thumbsdowns.json record

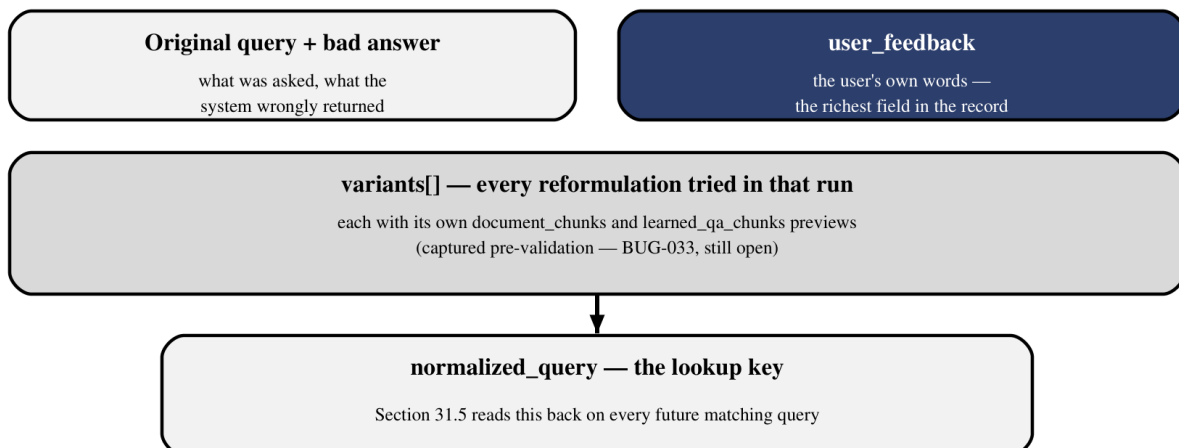


Figure 31.1 — A thumbsdown record preserves not just what went wrong, but every search angle that was tried in the run that produced the wrong answer.

Capturing every variant, not only the query that produced the final bad answer, is what makes Section 31.6's injection useful rather than merely accusatory. If a future run only knew *that* a question failed, the best it could do is try harder with no direction. Knowing *which specific reformulations* were tried and what each one retrieved gives a future run's `_PROCESS_INSTRUCTIONS` retry logic something concrete to avoid repeating and, by elimination, some sense of which angles remain unexplored.

Figure 31.1 lays out this full shape as four field groups, deliberately echoing Chapter 29.1's interaction-record anatomy — a thumbdown record is best understood as an interaction record's evidence and trajectory fields, carried forward and enriched with the one thing a plain interaction never has: the user's own explanation of what specifically was wrong.

31.4 Persisting to `user_thumbdowns.json`

`_mark_last_bad()` performs two logically related writes: flipping the last interaction's `quality` field, and appending the new thumbdown record. This project's real implementation wraps both in a single MongoDB transaction (ADR-048) specifically because a partial write — the interaction marked bad but no thumbdown record ever created, or vice versa — leaves the two files internally inconsistent with each other in a way that is hard to detect and worse to reason about later. In the original flat-file design this chapter's own era predates, the equivalent discipline is simpler but still real: write the thumbdown record *before* rewriting the interaction's quality field, so a crash between the two steps leaves the richer record intact even if the flag update is lost, rather than the reverse.

BUG-044 is the cautionary tale for this exact write: `_app_workflow/`'s first port of `_cmd_bad` called `_mark_last_bad(feedback=...)` without passing `variants` at all, so every thumbdown record was written with an empty `variants` array — Section 31.3's richest field, silently dropped. The record still existed, still had the feedback text, but had nothing for Section 31.6's injection to actually show the model about what was tried and failed. A record that is present but empty in its most important field fails more quietly than a record that never gets written at all.

31.5 Looking up prior thumbdowns by normalized query

`_find_thumbdowns_for_query()` normalizes the incoming query the identical way `_append_thumbdown()` normalized it at write time — lowercased, stripped — and returns every thumbdown record whose `normalized_query` matches exactly. This lookup runs once per new query, before the system prompt is built, so its result can feed directly into Section 31.6's injection.

```
@staticmethod
def _normalize_query(q: str) -> str:
    return q.lower().strip()

def find_thumbdowns_for_query(self, query: str) -> list[dict]:
    norm = self._normalize_query(query)
    return [d for d in self._thumbdowns.find({"normalized_query": norm})]
```

31.6 Injecting USER-FLAGGED PRIOR FAILURE

When ``find_thumbsdowns_for_query()`` returns one or more records, ``_build_system_prompt()`` (Chapter 26.2) builds a dedicated block naming exactly what went wrong last time and listing the reformulations already tried, placed in the **middle** of the prompt — after ``_ROLE_AND_RULES``, before ``_PROCESS_INSTRUCTIONS`` — as a passive "here is what happened" block. A second, shorter **active** block, built only from the feedback text itself, is appended after ``_PROCESS_INSTRUCTIONS``, closest to the generation point, framed as "PRIORITY — USER FEEDBACK MUST BE ADDRESSED."

This split — passive history in the middle, active priority last — is Chapter 26.1's recency discipline applied specifically to thumbsdown injection: the full historical context is useful but not the single most important thing the model needs to act on right now, while the distilled instruction to specifically target the user's flagged gap is exactly that, and earns the position recency bias weights most heavily.

31.7 Worked example: the ASD disambiguation case

``run_batch.py``'s Batch 1 script is the literal transcript of this mechanism working. Questions one through six ask variations of "What is ASD" and get confident autism-domain answers. Question seven is scripted as a ``bad`` command with the feedback: "I was actually asking about Adjustable Speed Drives, not autism. All your answers were about the wrong topic." Question eight re-asks the exact original question — "What is ASD and what are its main characteristics?" — and the ``USER-FLAGGED PRIOR FAILURE`` block built from question seven's feedback is now present in the system prompt for that retry.

Figure 31.2 traces this exact three-beat structure — six unchallenged wrong answers, one corrective thumbsdown, one corrected re-ask — because the shape generalizes well beyond this specific acronym collision. Any corpus mixing genuinely unrelated domains under overlapping vocabulary will eventually produce this identical pattern, and the fix is never better embeddings or a higher similarity threshold; it is exactly the out-of-band correction this chapter's mechanism exists to capture and replay.

Worked example: the ASD disambiguation thumbsdown

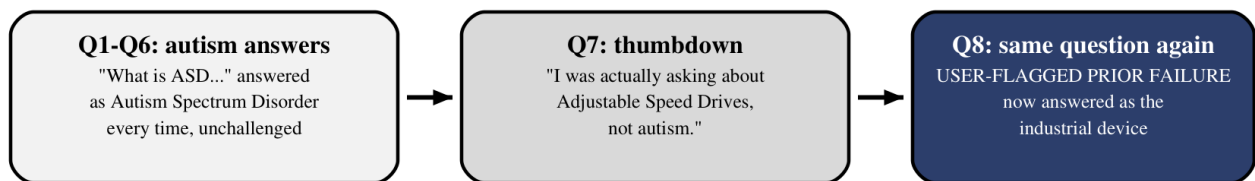


Figure 31.2 — Six autism-domain answers, one thumbsdown, one corrected re-ask — the mechanism this chapter builds, traced through a real scripted batch.

This worked example demonstrates the mechanism's actual value precisely because the underlying ambiguity — "ASD" genuinely meaning two unrelated things in this project's own mixed corpus — cannot be resolved by better retrieval alone. No amount of tuning ``RETRIEVAL_TOP_K`` or the similarity threshold fixes a genuinely ambiguous three-letter acronym; only a signal from outside the retrieval pipeline itself, the user's own disambiguating correction, can.

31.8 Content-vs-presentation feedback

Not every thumbdown carries the same kind of signal, and conflating the two kinds weakens both. Content feedback tells the system it retrieved or reasoned about the **wrong thing** — Batch 1's "I was actually asking about Adjustable Speed Drives" is a pure content correction, directly actionable by steering the next retrieval toward a different domain. Presentation feedback tells the system its answer was **about the right thing but shaped wrong** — Batch 13's real scripted feedback, "The answer was too generic. I wanted to know specifically about the control unit's fault detection role," is asking for more specificity on the identical topic, not a different topic entirely.

Both are legitimate, useful thumbdowns, and this project's ``_append_thumbdown()`` captures both identically — but Section 31.6's injection treats them identically too, which is a real limitation worth naming honestly rather than glossing over. A content correction gives the next retrieval a genuinely different query angle to try. A presentation correction gives the next retrieval almost nothing new to search for — the right chunks were probably already being retrieved — and the actual fix belongs more in how the draft is written from those chunks than in what gets searched for next.

31.9 Why "not structured enough" feedback can't help retrieval

Push the presentation-feedback case further and it exposes a structural boundary of this whole mechanism: feedback like "not structured enough" or "too generic" describes a property of the **generation** step, not the **retrieval** step, and ``_PROCESS_INSTRUCTIONS``'s retry path only knows how to do one thing in response to any INSUFFICIENT-style signal — call ``retrieve_documents`` again with different query angles. Re-retrieving cannot fix a formatting or specificity complaint about chunks that were already the right chunks; it can only ever change **what** gets found, never **how** the found material gets written up.

Common pitfall — Routing every complaint through the retrieval retry

A thumbdown-driven retry loop that only knows how to search differently will search differently even when the actual defect was in generation, not retrieval. Recognizing presentation-only feedback and routing it toward a drafting-instruction adjustment, rather than another retrieval round, is a real gap this project's mechanism leaves open rather than one it already closes.

31.10 Exact matching vs. fuzzy/semantic matching

Section 31.5's lookup is exact-string matching on normalized text — the identical trade-off Chapter 30.2 made for the failed-variants blocklist, and for the identical reason. A thumbdown recorded for "what is asd" will not surface for a later "what does asd mean," even though a human would recognize them as the same question. Research topic 38's finding — cosine similarity on sentence embeddings reliably clusters genuine paraphrases above a 0.95 threshold, while Jaccard token overlap misses them — is directly applicable here and was, in fact, the project's own basis for choosing cosine over Jaccard for query-variant deduplication elsewhere in the pipeline.

Upgrading Section 31.5's lookup from exact match to embedding-similarity match is a real, well-precedented option — not a hypothetical one — but it is not free: every new query would need an embedding call before the thumbdown lookup could even run, adding latency to a step that currently costs nothing beyond a dictionary or index lookup, and a similarity threshold introduces exactly the same precision/recall tuning burden Chapter 36C dedicates a full chapter to. Exact matching is the conservative default this project shipped with; fuzzy matching is the documented, well-understood upgrade path for a deployment where near-duplicate phrasing turns out to be common enough to justify the added cost.

Chapter 30's silent failures and this chapter's explicit ones both feed the same downstream question Chapter 32 finally answers: given a growing ledger of what went wrong, what does a system do with

everything that went *right*? Distillation is where this project's self-learning layer stops merely avoiding repeated mistakes and starts actively compounding its own verified successes.

It is worth closing on what these two chapters, taken together, buy a deployment. Neither mechanism makes the underlying retrieval or generation smarter in any general sense — Chapter 30's blocklist and this chapter's thumbdown injection are both narrowly scoped, exact-match, session-spanning corrections to specific, previously-observed failures. Their value is not breadth but reliability: the same mistake, once caught, does not need to be caught again. That is a modest claim, and it is exactly the honest, auditable kind of self-improvement Chapter 28 argued memory injection could deliver without pretending the model itself had learned anything at all.